

Freenet

A Peer-to-Peer Platform for Real-Time Decentralized Applications

Idempotent Commutative Monoids and Adaptive Small-World Routing

Ian Clarke

ian@freenet.org

<https://freenet.org>

May 24, 2026

Abstract

Freenet is a peer-to-peer platform for building real-time decentralized applications without global consensus. Its core abstraction is a *contract*: a WebAssembly module that defines both the public state of an application and the merge semantics used to reconcile replicas. Rather than imposing a single consistency model, the platform requires each contract to expose an idempotent commutative merge operation (equivalently, a join-semilattice) and an application-defined summary/delta protocol for efficient synchronization.

Each contract has a location on a one-dimensional ring derived from the hash of its code and parameters, and is replicated on peers near that location. Requests are routed using a per-neighbor, distance-indexed performance model rather than topological distance alone. For private state, the platform provides *delegates*: local WebAssembly agents that hold secrets on the user's device and interact with contracts and other software through attributed messages.

This architecture targets decentralized applications such as group chat, collaborative documents, social feeds, and reputation systems, where availability and application-defined convergence are preferable to global linearizability. The paper describes the architecture as currently implemented in the freenet-core reference runtime, summarizes a 24-hour live-network measurement of routing path lengths, and is explicit about which mechanisms are deployed, which remain experimental, and which are open problems.

1 The Problem

The modern web is built on infrastructure owned by a small number of companies. Group chat, social feeds, collaborative documents, identity, and reputation all run on it. So does the power to moderate, surveil, and remove the people who use it. This paper describes a peer-to-peer platform aimed at providing the same kinds of services without the same single points of control.

Two broad families of decentralized systems have been deployed at meaningful scale, and neither is a good substrate for interactive applications.

Content-addressed storage. The original Freenet (Clarke et al., 2001) and IPFS (Benet, 2014) provide name-by-hash access to immutable data. Mutable state is whatever the application chooses to layer on top. This is a poor fit for any system whose primary action is mutation: chat rooms, document editing, social feeds. In practice the application reinvents a consistency story per use case, often inconsistently across clients.

Blockchains. Ethereum (Wood, 2014) and successor smart-contract platforms provide programmable mutable state, but their consistency model is global linearizable consensus. Every state transition is ordered with every other transition, and replicated by every participating node. The latency, throughput, and replication cost that follow are reasonable for value-transfer and authoritative ledgers, but they are too expensive for the interactive applications that constitute the bulk of the web.

The gap between these is large. A chat room does not need a global linear order on every message; it needs replicas that converge to the same content. A reputation system does not need every reader to download every signed record; it needs efficient summary-driven sync. A collaborative document does not need consensus on each keystroke; it needs an algebra of merge that the application defines.

This paper describes a peer-to-peer platform built around that observation. Applications carry their own merge semantics, expressed as sandboxed WebAssembly. The platform handles routing, replication, synchronization, transport, and sandboxing generically over whatever algebra the application supplies, alongside a separate primitive (the delegate) for private state and sensitive operations that should not be replicated. The result targets a class of decentralized applications that has historically been awkward to build on either content-addressed or blockchain substrates.

2 Design Thesis

The platform’s central bet is to separate *what merges* from *how it propagates*. The application defines the algebra of its state: an idempotent, commutative merge operation on byte-string state, packaged as a WebAssembly module. The platform defines the rest: where state lives on the network, how peers find each other, how they exchange differences efficiently, how updates fan out to subscribers, how secrets are held locally. The platform is generic over the algebra; the application carries the lattice.

This design choice is what distinguishes the system from both ends of the existing decentralized spectrum.

Against fixed-lattice CRDTs. Conventional Conflict-free Replicated Data Type (Shapiro et al., 2011) frameworks expose a closed catalogue of lattices (counters, sets, registers, sequences) to applications. Each application chooses from the menu. The platform here generalizes the menu: a contract supplies its own merge function, with the validity invariants and tiebreak rules its workload requires. The trade is that the runtime cannot statically verify the algebraic properties: a contract whose “merge” is not associative will not converge. The contract is content-addressed, however, so any deviation is attributable to a specific module that users can choose to use or avoid.

Against global consensus. Blockchain platforms order all state transitions globally. That is more than most decentralized applications need. The system here provides per-contract eventual consistency: each contract’s state converges across its own replicas without requiring agreement with the rest of the network on the order in which its updates were applied. This makes the interactive, conversational class of applications natural, at the deliberate cost of being unsuitable for applications that require a single global order.

Public state and private state are different problems. Real applications have both: a chat room is public, the user’s signing key is private. The platform makes the distinction first-class. Contracts hold the public, replicated state. Delegates hold the private, device-local

state. A clean interface between them, with attributed messages crossing a platform-enforced trust boundary, replaces the usual story of “each application figures out where to keep its secrets,” which has not worked well in practice.

The remainder of the paper describes the primitives that follow from this thesis (Section 3), how updates move through the network (Section 4), how data is located (Section 5), how trust is established and what it does not cover (Section 6), the current implementation status and open problems (Section 7), and the position in related work (Section 8).

3 Core Primitives

This section names the artifacts that appear in the rest of the paper and fixes notation.

3.1 Peers and the Ring

A *peer* is an instance of the Freenet Core running on some host. Each peer p has a *location* $\ell(p) \in [0, 1)$. Locations live on a one-dimensional ring with wrap-around distance

$$d(x, y) = \min(|x - y|, 1 - |x - y|).$$

A peer’s location is derived deterministically from its external network address; the exact derivation and its security implications are discussed in Section 6. Each peer maintains a set of *neighbor* connections, bounded between configurable minimum and maximum sizes (typically 25 and 200). Neighbor relations are reorganized over time by the topology rule described in Section 5.

3.2 Contracts

A *contract* $\mathcal{C}$ is a pair (code, params), where code is a WebAssembly module conforming to the platform’s contract interface and params is an opaque byte string fixed at instantiation. The contract’s *key* is

$$k(\mathcal{C}) = H(H(\text{code}) \parallel \text{params})$$

for a cryptographic hash H . The two-stage form lets a client that already holds $H(\text{code})$ derive the key for a new contract from new parameters without re-hashing the full WebAssembly module, which may be several megabytes. The contract’s *location* $\ell(\mathcal{C}) \in [0, 1)$ is derived from $k(\mathcal{C})$ in the same way as a peer location. Each contract has a *state* $\sigma \in \mathcal{S}_{\mathcal{C}}$, replicated across peers near $\ell(\mathcal{C})$ on the ring. The contract’s code defines $\mathcal{S}_{\mathcal{C}}$, the validity predicate $\text{valid}_{\mathcal{C}}$, the merge operation $\oplus_{\mathcal{C}}$, and the summary/delta synchronization functions (all detailed in Section 4). Contracts are public: their state is readable by any peer that can route to $\ell(\mathcal{C})$.

3.3 Delegates

A *delegate* is a WebAssembly module that runs on a single user’s device, inside the local Freenet Core. It holds private *secret state* (sandboxed by the core) and responds to messages from user interfaces, contracts, and other delegates. The core attaches a verified sender identifier to every inbound message, so a delegate can apply policy based on sender identity in addition to message content. Delegates are local; they do not appear on the ring. The trust boundary they enforce and the cross-device replication story are described in Section 6.

3.4 User Interfaces

A *user interface* is a web frontend (HTML, JavaScript, WebAssembly) delivered by a contract whose state encodes the application bundle. User interfaces run in the browser and communicate with the local core over a WebSocket API. They reach contracts and delegates only through that local core. They are not a distinct kind of network-visible artifact: from the platform's perspective, the bundle is just contract state with a particular MIME type.

3.5 Operations

A small set of operations connect these artifacts. We describe them informally here; their detailed lifecycles appear in Section 5.

PUT

Publish a contract: insert or update its state at the peers near $\ell(\mathcal{C})$.

GET

Retrieve a contract's current state by key.

UPDATE

Submit an update to a contract. The update is merged into existing replicas using the contract's merge function and propagated outward through the subscription tree.

SUBSCRIBE

Register interest in future state changes to a contract.

CONNECT

Establish a peering relationship with the network as a new node.

PUT, GET, UPDATE, and SUBSCRIBE all route over the ring; their destination is $\ell(\mathcal{C})$. CONNECT is the bootstrap operation by which a fresh peer joins the topology.

4 How Updates Move

This section describes the algebra each contract carries, the synchronization protocol that propagates it efficiently across replicas, and the subscription tree along which updates flow.

4.1 Contract State as an Idempotent Commutative Monoid

Definition 1 (Contract state algebra). *A contract $\mathcal{C}$ defines:*

1. a state space $\mathcal{S}_{\mathcal{C}} \subseteq \{0, 1\}^*$ of byte strings, with a validity predicate $\text{valid}_{\mathcal{C}} : \{0, 1\}^* \rightarrow \{\perp, \top\}$;
2. an identity element $e_{\mathcal{C}} \in \mathcal{S}_{\mathcal{C}}$ with $\text{valid}_{\mathcal{C}}(e_{\mathcal{C}}) = \top$;
3. a binary merge operation $\oplus_{\mathcal{C}} : \mathcal{S}_{\mathcal{C}} \times \mathcal{S}_{\mathcal{C}} \rightarrow \mathcal{S}_{\mathcal{C}}$, which is associative, commutative, and idempotent, with $e_{\mathcal{C}}$ as identity.

An idempotent commutative monoid is equivalently a join-semilattice; the partial order is $a \sqsubseteq b \iff a \oplus_{\mathcal{C}} b = b$. This is precisely the algebraic structure of a state-based CRDT (Shapiro et al., 2011). We use the monoid presentation because it foregrounds the operational property the platform relies on: peers can merge each other's state at any time, in any order, with any multiplicity, and arrive at the same result.

The validity predicate is checked at the boundary. A peer that receives a candidate state σ with $\text{valid}_{\mathcal{C}}(\sigma) = \perp$ rejects it and never merges it into its local replica. Validity typically encodes integrity invariants beyond what the type system enforces, for example “every record in the log carries a valid signature by an authorized key.”

An *update* is itself an element of $\mathcal{S}_{\mathcal{C}}$. When a peer holding state σ receives an update u , it computes $\sigma' = \sigma \oplus_{\mathcal{C}} u$, subject to validity. There is no separate notion of operation: the algebra of state is the algebra of change.

Proposition 1 (Convergence). *Let $u_1, \dots, u_n \in \mathcal{S}_{\mathcal{C}}$ be a finite multiset of valid updates and let $\sigma_0 \in \mathcal{S}_{\mathcal{C}}$ be a starting state. For any permutation π ,*

$$\sigma_0 \oplus_{\mathcal{C}} u_{\pi(1)} \oplus_{\mathcal{C}} \dots \oplus_{\mathcal{C}} u_{\pi(n)} = \sigma_0 \oplus_{\mathcal{C}} u_1 \oplus_{\mathcal{C}} \dots \oplus_{\mathcal{C}} u_n.$$

The argument is immediate from associativity and commutativity. Idempotence further makes duplicate delivery harmless. Together these properties are the sufficient conditions for the Strong Eventual Consistency theorem of Shapiro et al. (Shapiro et al., 2011): peers that have delivered the same set of updates converge regardless of order or repetition.

4.2 Worked Examples

Monotone counter. $\mathcal{S} = \mathbb{N}$, $\oplus = \max$, $e = 0$. The partial order is the usual $\leq$.

Last-writer-wins register. $\mathcal{S} = V \times \mathbb{N} \times H$, where V is the value type, $\mathbb{N}$ a Lamport-style timestamp, and H a tiebreaker hash. Merge is lexicographic max on the (timestamp, hash) pair.

Signed log. $\mathcal{S}$ is a finite set of records, each record carrying a signature by an authorized key. $\oplus = \cup$, $e = \emptyset$. Validity rejects any state containing a record whose signature does not verify against a key listed in params.

Observed-remove set. $\mathcal{S} = (E, T)$ where E is a set of (elem, id) pairs and T a set of ids that have been tombstoned. $\oplus$ is pairwise union; the materialized set is $\{\text{elem} : (\text{elem}, \text{id}) \in E, \text{id} \notin T\}$.

A bounded variant of the signed log (say, “keep the most recent N records”) is straightforward but requires care: the eviction function must be a deterministic function of the merged set, not of the order in which records arrived, or idempotence is lost. We treat this as a contract-author obligation rather than a platform-enforced property.

4.3 The Generalization over Fixed-Lattice CRDTs

A conventional CRDT runtime offers a closed catalogue of lattices. Each application chooses one. The contract interface inverts this: the application supplies the lattice. Two consequences follow.

First, the platform cannot statically verify that a contract’s $\oplus_{\mathcal{C}}$ is associative, commutative, and idempotent. A contract that violates these properties will exhibit non-converging replicas. Because the contract is content-addressed, however, any deviation is attributable to a specific module; a contract that does not converge is bug-compatible with the rest of the network only insofar as users choose to deploy it.

Second, an application can choose merge semantics tuned to its workload. The consistency requirements of a chat room, a reputation contract, and a public file feed are very different, and bundling all of them into a single lattice family would either force one to pay another’s cost or leave several of them unimplementable.

4.4 Summary/Delta Synchronization

Pairwise merge is sufficient for convergence but, on its own, would require peers to ship full state when reconciling. For non-trivial contracts this is prohibitive. The platform therefore extends the contract interface with three additional functions that allow peers to exchange only the difference between their states.

Definition 2 (Sync interface). *In addition to the algebra of Definition 1, a contract provides*

$$\begin{aligned}\text{summarize}_C &: \mathcal{S}_C \rightarrow \Sigma_C, \\ \text{getDelta}_C &: \mathcal{S}_C \times \Sigma_C \rightarrow \Delta_C, \\ \text{applyDelta}_C &: \mathcal{S}_C \times \Delta_C \rightarrow \mathcal{S}_C,\end{aligned}$$

where Σ_C and Δ_C are contract-defined byte-string types.

The summary type Σ_C is intended to be much smaller than full state. For a signed log it might be the set of record identifiers, or a Merkle root over them; for a counter it might be the counter's value; for a large observed-remove set it might be a Bloom filter, or a pair of Bloom filters over E and T .

For the two-step protocol below to be useful, the contract author is required to ensure that getDelta produces something that, when applied, dominates the other peer's state in the partial order:

Property 1 (Sync soundness). *For all $\sigma_A, \sigma_B \in \mathcal{S}_C$, let $s = \text{summarize}_C(\sigma_B)$ and $\delta = \text{getDelta}_C(\sigma_A, s)$. Then*

$$\text{applyDelta}_C(\sigma_B, \delta) \oplus_C \sigma_A = \text{applyDelta}_C(\sigma_B, \delta).$$

Equivalently, applying δ at σ_B brings σ_B at least as far up the lattice as σ_A would have if merged in full. The platform does not verify this property; the contract is responsible for it.

4.5 The Two-Step Sync Protocol

Two peers A and B reconcile their states for a contract C as follows:

1. A computes $s_A = \text{summarize}_C(\sigma_A)$ and sends s_A to B .
2. B computes $\delta_B = \text{getDelta}_C(\sigma_B, s_A)$ and sends δ_B to A . A applies the delta: $\sigma_A \leftarrow \text{applyDelta}_C(\sigma_A, \delta_B)$.

The protocol is symmetric: B runs the same exchange in the other direction. In practice the two directions are interleaved over a single connection. Bandwidth is bounded by $|s_A| + |s_B| + |\delta_A| + |\delta_B|$, each of which is application-defined.

4.6 Malformed Summaries and Malicious Deltas

The honest-but-stale case is the easy one. Two cases require more care.

Approximate summaries. A summary type such as a Bloom filter has false positives: B may believe A already has a record that in fact A does not. In the bare protocol, the record is then not included in the delta and the two replicas remain divergent. Contracts that use approximate summaries must therefore either fall back to a more precise second round when divergence is suspected, or accept the false-positive rate as part of their convergence story (relying on later rounds to repair). The platform offers no built-in fallback; the contract designs the round structure.

Malicious peers. A peer that supplies a deliberately malformed summary, or returns a delta that violates the validity predicate, is treated like any other source of invalid state. The receiver applies valid_C to the merged result; if the result fails validity, the delta is rejected. A more subtle attack is for a peer to supply a delta that is valid but withholding (claiming to be up to date when it is not). The protocol cannot distinguish such a peer from one that has simply not yet received the missing updates; the replication factor across multiple peers near $\ell(C)$ is the structural defense, not the sync protocol itself.

4.7 Subscription Trees

Once a contract is published, peers near $\ell(C)$ host its state and additional peers register as subscribers. A subscription forms a soft-state link from the subscriber back toward the contract's home location; collectively these links form a tree rooted near $\ell(C)$. An update arriving at any node of the tree is merged into local state and propagated outward along the tree, with each pairwise hop using the summary/delta protocol above.

Because $\oplus_C$ is commutative, simultaneous updates injected at multiple points of the tree do not require coordination. They propagate along the edges, meet at internal nodes, and merge; the order in which different updates arrive at a given node does not affect its final state. Subscriptions are lease-based: in the deployed configuration a lease lasts 8 minutes and is renewed every 2 minutes by a background task. A subscriber whose lease expires must re-subscribe. The leasing makes the tree self-healing: orphaned branches die out within a small multiple of the lease period.

4.8 Relationship to Other Mechanisms

Versus operation-based CRDTs. An operation-based CRDT propagates individual operations and relies on a causal-broadcast layer to ensure exactly-once, causally ordered delivery. The protocol here propagates state and tolerates arbitrary loss, reordering, and duplication; the merge function does the work that causal broadcast would otherwise have to.

Versus Merkle anti-entropy. Distributed key-value stores commonly use Merkle-tree anti-entropy (DeCandia et al., 2007) to discover differences between replicas. That is a special case of the protocol here in which the summary is a hash tree and the delta is the subtree the other replica is missing. A contract whose data structure is amenable to a more compact summary (a list of identifiers, a counter value) pays a single round trip rather than $O(\log n)$ of them.

Versus rsync. `rsync` (Tridgell, 1999) reconciles files by exchanging block-level checksums. It is generic but oblivious to application structure. Summary/delta is the same shape of protocol lifted to a level where the application defines the granularity.

Versus delta-state CRDTs. The closest formal analogue is the delta-state CRDT literature (Almeida et al., 2018), which ships only changes between replicas of a fixed-lattice CRDT. The differences here are that the lattice and the delta-encoding are both supplied by the contract, and the result is embedded in a small-world routing layer rather than a fully-connected replica set.

5 How Data Is Found

A request for contract $\mathcal{C}$ originates at some peer and must reach a peer that hosts $\mathcal{C}$'s state. This section describes the ring topology, the rule that constructs it, the routing logic that uses it, and the bootstrap by which a new peer joins. The section closes with a 24-hour measurement of routing path lengths on the live network.

5.1 Locations and the Ring

A peer's location $\ell(p)$ is computed by hashing its external network address into $[0, 1)$. A contract's location $\ell(\mathcal{C})$ is similarly derived from its content-addressed key. Distance is the wrap-around metric of Section 3.

This is the simplest workable arrangement, and its trade-offs are worth being explicit about. Locations are stable across restarts that preserve the peer's address but change when a peer is rehomed, when NAT mappings shift, or when a peer is reachable on multiple addresses. An adversary in control of many addresses (a cloud subnet, an IPv6 range, a small botnet) can grind locations and concentrate them in chosen regions of the ring. The platform does not attempt to solve Sybil resistance at this layer; defenses are application-level (signed contributions, reputation contracts, capacity-limited delegation) and are discussed in Section 6.

5.2 Topology by Acceptance Rule

The small-world property of the ring is constructed by a single rule. A new peer p joining the network sends a **CONNECT** message to a known *gateway* peer, with the target field set to $\ell(p)$ itself. The message is forwarded by the rule:

*If I have a neighbor closer to the target than I am, forward the **CONNECT** to that neighbor; otherwise accept the connection.*

We call this the *accept-only-at-terminus* rule. Its consequences:

Local clustering. The accepting peer is, by construction, near $\ell(p)$ on the ring. Each join reliably produces a connection between two peers that are close on the ring.

Long-range links from ordinary traffic. Every **PUT**, **GET**, **UPDATE**, and **SUBSCRIBE** is forwarded greedily toward a contract location chosen by application logic. Any of these requests, when terminating, can leave a fresh connection between the originating peer and a peer at an essentially arbitrary ring distance. The application-driven distribution of contract locations is what supplies the long-range edge distribution, not an explicit Kleinberg (Kleinberg, 2000) sampling step.

Bounded fan-out. A peer accepts only when it cannot forward closer. Once its neighborhood is densely packed, the local terminus typically lies elsewhere, and admission control bounds the connection count between configurable minimum and maximum.

5.3 Greedy Forwarding

A request from peer p to target location t proceeds as follows. Let $N(p)$ be p 's current set of neighbors. If some neighbor $q \in N(p)$ is strictly closer to t on the ring than p itself, forward the request to the closest such q ; otherwise p is a terminus for the request and serves it locally.

Each request carries a *hops-to-live* (HTL) counter that decrements on each forward. The deployed maximum is 10. Above a configurable threshold (HTL > 7 by default) the next hop is chosen by random walk rather than greedily. The fallback ensures the request leaves the immediate neighborhood of a peer that happens to be very near t but has not cached the contract yet, and contributes path diversity that helps the topology mix.

5.4 Adaptive Routing

Greedy-by-distance assumes the topologically closest neighbor is also the operationally fastest path. In a real peer-to-peer network this is wrong often enough to matter: neighbors vary by an order of magnitude or more in bandwidth, RTT, and uptime. The router therefore picks the next hop not by ring distance alone but by predicted total cost.

For each neighbor q , the local peer maintains a history of routing events of the form (d, τ, b, f) , where d is the ring distance from q to the request's target, τ is the observed time-to-first-byte, b is the observed transfer rate, and f is a failure indicator. From this history three estimators are fit:

$$\hat{\tau}_q(d), \quad \hat{p}_q(d), \quad \hat{b}_q(d),$$

the predicted latency, failure probability, and throughput as functions of distance-to-target. Each is fit by *isotonic regression* (Barlow et al., 1972; de Leeuw et al., 2009), the standard non-parametric estimator for a monotone function.

The choice of isotonic regression encodes a structural prior we are willing to commit to (latency and failure are non-decreasing in distance-to-target, on average) without committing to a parametric family for the shape of the curve. The pool-adjacent-violators algorithm fits each curve in time logarithmic in history size; old events age out so the model tracks current conditions.

Given a target t for a new request, the router selects the neighbor that minimizes an expected-cost score combining the three estimators. A neighbor with too little history falls back to greedy-by-distance until enough events have accumulated.

5.5 Lifecycles

The five operations of Section 3 are realized as transaction state machines coordinated by the local core. Each is assigned a transaction id, tracked through completion, and supports timeout, retry, and composite parent/child relationships.

CONNECT. As above. A successful **CONNECT** establishes a single neighbor relationship. A joining peer typically issues several at slightly different target locations to populate its initial neighbor set; subsequent topology evolution is driven by ordinary request traffic.

PUT. A **PUT** carries a contract $\mathcal{C}$ and an initial state σ_0 . The originating peer routes the request toward $\ell(\mathcal{C})$. The first peer with capacity at or near $\ell(\mathcal{C})$ caches $\mathcal{C}$, checks $\text{valid}_{\mathcal{C}}(\sigma_0)$, and accepts the publish. Nearby peers receive the state through the same path and replicate it.

GET. A **GET** carries a key k . The originating peer routes toward the corresponding location. Any peer along the path that currently hosts the contract may answer with its current state σ and parameters; the response travels back along the request path, optionally caching at intermediate peers. The requester can verify on arrival that the served code and parameters hash to k . State itself is not directly verifiable from the key (the key is derived from code and

params, not from state), and is therefore trusted to the extent that the application’s validity predicate accepts it. Replication across several peers near $\ell(\mathcal{C})$ is the structural defense against a single peer serving a stale or doctored state.

UPDATE. An UPDATE carries a key k and a candidate state $u \in \mathcal{S}_{\mathcal{C}}$. It is routed toward $\ell(\mathcal{C})$. Each peer along the path that already hosts the contract merges u into its local copy subject to the validity check, then propagates the change outward through the subscription tree (Section 4).

SUBSCRIBE. A SUBSCRIBE establishes a lease on future updates. The subscriber routes the request toward $\ell(\mathcal{C})$ and the first hosting peer responds; intermediate peers along the path record the subscription. Subscriptions are leased and renewed periodically (Section 4).

5.6 Live-Network Measurement

We summarize routing behavior on the live network over a 24-hour window ending 2026-05-24. Network size in the window was 443 distinct active peers in the last 10 minutes, with 604 distinct peer identifiers seen across the full 24 hours (the difference attributable to churn). All peers run with telemetry enabled in the current deployment.

For each operation type we compute a per-transaction *path length*: the number of distinct peers seen emitting the corresponding `*_request` event for that transaction id. Path length so defined is the number of peers visited (hops + 1). Statistics over operations whose telemetry reached completion in the window:

Operation	n	mean	median	p95	p99	max
GET	1,302	6.84	7	11	11	11
PUT	158	6.01	7	10	10	10
UPDATE	386	1.88	2	2	2	5
SUBSCRIBE	5,359	1.54	1	4	5	12

UPDATE and SUBSCRIBE path lengths are short because most such requests propagate through pre-existing subscription trees rather than fresh greedy routes. The relevant numbers for the routing layer are GET and PUT. For $N \approx 443$, $\log_2 N \approx 8.8$; the median GET path length of 7 (6 hops) sits comfortably below this bound, and the p95 of 11 sits at the HTL ceiling. That ceiling truncates the long tail: routes that would otherwise terminate at 12 or more peers are dropped rather than appearing in the table.

We are deliberately modest about what this single data point shows. A single 24-hour window at a single network size is not a scaling study; a real test of the $O(\log n)$ claim would scan N across orders of magnitude. What the data does show is that on the deployed network, with the deployed routing rule, the typical GET reaches its destination in single-digit hops, consistent with the small-world expectation.

6 Security and Trust Boundaries

The platform has several layers, and each authenticates and trusts different things. This section is the consolidated story of what guarantees the platform actually provides, where the boundaries are, and what it does *not* attempt to defend against.

6.1 Transport: Hop-by-Hop Encryption and Authentication

A connection between two neighboring peers is established by an authenticated ephemeral Diffie-Hellman handshake (X25519). The handshake exchanges symmetric keys used to encrypt subsequent traffic with AEAD constructions: AES-128-GCM by default, with ChaCha20-Poly1305 as an alternative for platforms without AES hardware acceleration. The handshake itself is encrypted with the recipient’s long-term identity key, which doubles as a defense against unsolicited probes: a peer that has not received a properly-encrypted hello does not respond at all, making the port effectively invisible to a passive scanner.

This authentication is *hop-by-hop*. A request that traverses peers $A \rightarrow B \rightarrow C \rightarrow D$ is authenticated by A to B , by B to C , and by C to D . There is no transport-level guarantee that the message body at D is the body A originally sent: an intermediate peer can drop, reorder, or modify the payload before forwarding.

6.2 Application Layer: End-to-End Authentication via Validity Predicates

End-to-end integrity is the contract’s responsibility. A contract’s validity predicate valid_C runs at every peer that touches the state. Update messages typically include a signature by an application-authorized key over the update content; the validity predicate verifies the signature and rejects the update otherwise. Because valid_C is itself part of the contract’s content-addressed code, a peer cannot serve a state that fails validity without being caught by every receiving peer that runs the same code.

The combination is the platform’s substitute for a single end-to-end transport channel. Transport gives hop-by-hop confidentiality and authentication of neighbors. The contract gives end-to-end semantic integrity of state.

6.3 NAT Traversal and Bootstrap

Most peers operate behind consumer NAT and do not have a stable inbound address. The transport supports hole-punching: when two peers want to connect and at least one is behind a NAT, both peers send hello packets toward each other’s observed external addresses simultaneously (Ford et al., 2005). The gateway, which is required to have a public IP, acts as an introducer that reports each side’s externally-observed address.

A peer’s location is derived from this externally-observed address. The implications are discussed in the next subsection.

6.4 Location Assignment: A Known Sybil and Grinding Surface

Hashing the external address into $[0, 1)$ is a deliberately simple location-assignment scheme, and it is not Sybil-resistant. An attacker controlling a subnet, an IPv6 range, or a small cloud account can grind locations and place many peers in a chosen region of the ring. Within that region they can serve targeted contract requests with selected (or no) responses, and they can crowd out honest peers from acceptance under the topology rule.

The platform does not solve this at the location layer. Two architectural mitigations operate at higher levels.

First, the contract layer’s validity predicates make state-doctoring detectable: a malicious peer that returns a state with an invalid signature is rejected by every honest receiver. The attack surface this leaves open is selectively serving stale (but still valid) state, not actively forging state.

Second, the platform expects identity, reputation, and contribution-control to be built as application-layer contracts (see Ghost Keys (The Freenet Project, 2026a) for an example of a contract-mediated anonymous identity scheme). A contract that posts authoritative information can require its writers to hold reputation tokens issued by another contract; an inbox contract can demand a small proof of work; a community forum can require introductions from existing members. The platform’s role is to provide a substrate on which these mechanisms can be built, not to embed any particular one in the routing layer.

We treat the absence of a built-in Sybil defense as a deliberate scope decision, not an unintended gap. A platform-level scheme tends to either over-fit (assuming a specific economic or identity model that not all applications share) or under-deliver (a generic mitigation that any sufficiently-resourced adversary defeats).

6.5 Delegates: A Trust Boundary Around Secrets

Contracts hold public state. Applications also have private state (a user’s signing keys, a private contact list, the decryption material for an end-to-end encrypted inbox) that should never be replicated and should be accessible only through controlled interfaces. The platform’s primitive for this is the delegate.

A delegate is a WebAssembly module running inside the local Freenet Core on a user’s device. It holds private secret state, sandboxed by the core. Other components (user interfaces, contracts, other delegates) can interact with it only by sending messages. The core attaches a verified sender identifier to each inbound message, so the delegate’s policy can be “only respond to messages from UI X, originating from contract Y.”

The trust boundary is therefore platform-enforced rather than conventional. Traditional in-process encapsulation (a “private” field on an object) is honored only by code that chooses to honor it; the same address space affords access to memory, file system, and undocumented APIs. A delegate’s secret state is not addressable from any other code on the device. The only way to extract a secret from a delegate is to convince the delegate, on the basis of sender identity and message contents, that releasing the secret is allowed.

Worked example: signing without exporting a key. A chat application’s UI needs to send signed messages. Without delegates, it must hold the private key. With delegates, the UI sends a (sign, payload) message to a key-manager delegate; the delegate verifies the sender, signs the payload, and returns the signature. The private key never crosses the boundary. A bug or compromise in the UI can produce unwanted signed payloads but cannot exfiltrate the key itself; the delegate is also free to apply rate limits or user confirmation prompts before signing.

Cross-device replication. A user typically runs the “same” delegate on multiple devices. The delegates synchronize through a shared-secret contract: the contract holds an encrypted, replicated representation of the shared private state, and only the delegates with the symmetric key can decrypt it. The contract’s merge function provides convergence; the delegate provides confidentiality.

6.6 Resource Accounting and What It Doesn’t Cover

Each peer enforces user-configured limits on bandwidth (token-bucket rate limiting at the transport layer), storage (an LRU cache of hosted contracts with a per-peer budget), CPU, and memory (per-contract WebAssembly fuel and explicit memory bounds, enforced by Wasmtime (Bytecode Alliance, 2026)). Excessive consumption by a neighbor results in disconnection.

This protects the peer against a single misbehaving neighbor. It does not protect against coordinated traffic from many neighbors, against adversarial DoS that uses many cheap identities to fill local queues, or against a network-level abuse story that survives across peer identities. Defenses in that regime probably require some form of accounting that persists across identities, which is one of the explicit open problems in Section 7.

7 Implementation Status and Open Problems

This section describes the reference implementation, makes the implementation status of each major mechanism explicit, and lists the architectural questions we view as open.

7.1 Implementation

The Freenet Core is implemented in Rust as an asynchronous `tokio`-based runtime. Contracts and delegates execute inside Wasmtime (Bytecode Alliance, 2026); both run under per-instance fuel limits and explicit memory bounds so that a runaway or malicious module cannot exhaust the host. Contract state is persisted through a pluggable store layer with implementations on top of `redb` (Berner, 2026) and SQLite.

The runtime is organized as a single central event loop, a `tokio::select!` over inbound transport packets, client API events, operation timers, and background tasks, that dispatches into per-operation state machines. The single-coordinator design simplifies reasoning about partial state but limits the throughput of a single peer to what one event-loop thread can sustain. So far this has not been the binding constraint in deployment.

Deterministic simulation testing. A peer-to-peer system is hard to test by ordinary means: many bugs in routing, congestion, or convergence appear only in specific multi-peer interleavings that are difficult to reproduce. The Core ships with a deterministic simulation harness built on Turmoil (Tokio Project, 2026): virtual time, a seeded global RNG, an in-memory socket implementation, and fault-injection knobs for loss, latency, partitions, and reorderings. Tests run the entire network in a single deterministic process and replay bit-for-bit from a seed. Most non-trivial routing, subscription, and contract-execution bugs we have found were first reproduced in simulation.

Content-addressed components. Contracts, delegates, and UI bundles are all addressed by hash. This pushes a lot of complexity off the platform: there is no version negotiation, no upgrade protocol, no need for the platform to understand application semantics during deployment. A new version is published under a new key; the old key continues to work as long as anyone hosts it.

7.2 Implementation Status of Each Mechanism

The following table is the current honest status of the mechanisms described in this paper. “Deployed” means present in the production runtime and exercised on the live network; “Implemented” means present in the runtime but not yet broadly exercised at scale; “Experimental” means available as a build-time option but not in production; “Open” means designed or partially designed but not implemented.

Mechanism	Status	Notes
Small-world ring topology	Deployed	Live network ≈ 440 active peers
Greedy forwarding + HTL fallback	Deployed	HTL cap = 10, random walk above 7
Accept-only-at-terminus topology rule	Deployed	
Adaptive isotonic-regression routing	Deployed	Per-neighbor distance-indexed model
Summary/delta synchronization	Deployed	Per-contract, application-defined
Subscription trees with leases	Deployed	8-min lease, 2-min renewal
Contract WebAssembly execution	Deployed	Wasmtime, fuel + memory limits
Delegates (local, secret state)	Deployed	Used by River chat (The Freenet Project, 2026b)
Delegate cross-device sync via shared-secret contract	Implemented	
NAT hole-punching transport	Deployed	UDP, X25519 + AEAD
Fixed-rate congestion control	Deployed	Production default
LEDBAT++ congestion control	Experimental	Did not meet our requirements in initial trials; not in production
BBR-style congestion control	Experimental	Build-time option, not the default
Deterministic simulation testing	Deployed	Turmoil-based (Tokio Project, 2026)
Per-peer resource limits (bandwidth, storage)	Deployed	User-configured budgets
Sybil-resistant peer identity / location	Open	Defenses are application-layer (see Section 6)
Network-level incentive / abuse layer	Open	Application-layer reputation contracts are the intended mechanism

The single biggest gap between this paper’s narrative and the deployed system is congestion control. The platform supports pluggable congestion controllers but the deployed default is a token-bucket fixed-rate scheme. LEDBAT++ and BBR variants exist in the codebase as alternatives, but neither has yet been adapted to the workload satisfactorily. Real congestion control across heterogeneous peer-to-peer links remains open work.

7.3 Where the Platform Fits

The combination of contract-defined merge, summary/delta sync, small-world routing, and delegates is well-matched to applications whose consistency requirements are conflict-mergeable in some application-defined sense. Group chat, collaborative documents, social feeds, reputation systems, asynchronous email-like inboxes, and identity directories are all in the well-fit class. A working chat implementation (The Freenet Project, 2026b) and an anonymous identity scheme (The Freenet Project, 2026a) are deployed against the live network.

7.4 Where the Platform Does Not Fit

Several application classes do not fit the model gracefully, and we want to be explicit about them.

Strictly linearizable transactions. A defense against double-spending fungible tokens requires that two attempts to spend the same balance cannot both succeed. The merge model cannot express “one of these updates is rejected unconditionally”; merge is total. Applications that need a strong-consistency primitive must either contain the inconsistency inside a contract

whose validity predicate detects and rejects the second update (which forces the conflict into application code) or layer an auxiliary consensus mechanism on top of the platform.

Abuse-resistant open posting. A purely-open contract is simple to write as a set under union and equally simple to flood. Practical contracts in this class restrict writes to signed records by approved keys, charge a proof-of-work fee per update, or require credentials issued by a reputation contract. The platform provides no built-in spam defense.

Coordinated DoS. Per-peer resource limits protect a peer against a single misbehaving neighbor, not against coordinated traffic from many neighbors using cheap identities. Defenses here probably require accounting that persists across identities, which is the open incentive layer noted above.

7.5 Open Problems

We treat the following as genuinely open and intend them as subjects of future work or empirical evaluation.

1. **Scaling characterization.** The single 24-hour data point in Section 5.6 is consistent with logarithmic-diameter routing at the deployed network size. A real characterization requires scanning N across orders of magnitude. We do not currently have the deployment to do this; controlled simulation studies are an interim alternative.
2. **Failure modes of contract-defined merge.** A contract whose merge is not actually associative or idempotent does not converge; the platform's handling of this is currently to allow the divergence to be observable but not to take corrective action. Better diagnostics, and where feasible contract-level invariants checked at the platform layer, are open work.
3. **Congestion control.** As above. The current fixed-rate default is a deliberate choice not to ship something we cannot tune correctly, not an endorsement of fixed-rate as the right answer.
4. **Sybil-resistant identity.** The application-layer approach has examples (Ghost Keys) but no clean general story; this is the recurrent question for every decentralized platform, and we do not claim to have solved it.
5. **Incentive / abuse layer.** How a network as a whole resists coordinated denial of service without a centralized authority, while still allowing low-friction joining, is the work that determines whether the network grows past a small community.

8 Related Work

Distributed hash tables. Chord (Stoica et al., 2001), Pastry (Rowstron and Druschel, 2001), and Kademlia (Maymounkov and Mazières, 2002) established the structured-overlay paradigm the ring inherits. These systems organize peers by hashed identifiers in a ring or XOR-metric space and provide $O(\log n)$ lookup with carefully maintained routing tables. Their consistency model is fundamentally immutable: a put-then-get returns the published value. The platform here replaces the immutable cell with a mutable, contract-defined one and replaces explicit finger-table maintenance with the implicit topology construction of Section 5.

Content-addressed storage. IPFS (Benet, 2014) popularized hash-based naming as a deployment substrate for web content, with a peer-to-peer block exchange (Bitswap) underneath. IPFS is also fundamentally immutable, with mutability provided externally (IPNS, smart contracts) and consistency left to applications. The choice here is to put mutability and consistency inside the platform via the contract abstraction; this is strictly more opinionated.

CRDTs. The theory of Conflict-free Replicated Data Types (Shapiro et al., 2011) is the algebraic foundation the contract layer builds on. The contract interface realizes the state-based (CvRDT) flavor with two extensions: the lattice is application-supplied rather than chosen from a catalogue, and the summary/delta protocol generalizes the implicit “ship the whole state” protocol of vanilla state-based CRDTs. Antidote and AntidoteDB (Akkoorath et al., 2016) are the closest comparison points from the database side, both being CRDT databases for the geo-distributed setting, but they expose a fixed CRDT vocabulary rather than letting applications define their own.

Anti-entropy and delta sync. Dynamo (DeCandia et al., 2007) and its descendants used Merkle anti-entropy to reconcile replicas. Bayou (Terry et al., 1995) pioneered an operation-log approach with application-defined merge procedures and is conceptually the closest historical analogue to contracts. Delta-state CRDTs (Almeida et al., 2018) formalize a state-based protocol that ships only changes between replicas, very much in the same shape as Section 4; the differences here are that the lattice and the delta encoding are both supplied by the contract, and the result is embedded in a small-world routing layer rather than a fully-connected replica set.

Small-world routing. Kleinberg’s 2000 result (Kleinberg, 2000) is the theoretical underpinning. The original Freenet (Clarke et al., 2001) predated and informally anticipated some of it. The accept-only-at-terminus topology construction is, to our knowledge, an unusual variant: it builds the long-range link distribution from request traffic rather than from explicit sampling. We are not aware of a prior theoretical analysis of routing performance under that construction.

Performance-aware routing. Adaptive routing in peer-to-peer overlays has a long history. Isotonic regression specifically has been used as a calibration tool in machine learning (de Leeuw et al., 2009); as far as we are aware, it has not previously been applied as the per-neighbor performance model in a structured overlay’s forwarding decision.

Blockchain platforms. Ethereum (Wood, 2014) and successor smart-contract platforms provide programmable consistency, paid for by a globally-linearized replicated state machine. The platform here relaxes the global linearization in favor of per-contract eventual consistency with application-defined merge. The trade is that a different class of applications is natural: the interactive, conversational ones, at the cost of being unsuitable for applications that fundamentally require a single global transaction order.

Federated systems. Matrix (The Matrix.org Foundation, 2026) is the closest non-peer-to-peer system in application surface. It is server-based and federated, and its state-resolution algorithm reconciles divergent room states by a deterministic application-defined rule. The platform here pushes the same idea into a fully peer-to-peer setting and generalizes the merge to arbitrary idempotent commutative monoids.

The original Freenet. The original Freenet (Clarke et al., 2001) was anonymous, content-addressed, immutable file storage from 2001. Its routing heuristic (forward to the peer whose previous successful routes most resemble the requested key) was an early precursor to small-world greedy routing, but it operated over a routing table that had no explicit notion of location, and the network’s data model was a single immutable blob per key. The platform described here preserves the small-world intuition and the content-addressed naming and replaces almost everything else: explicit locations, contract-defined mutable state, anonymity moved out of the core into application-layer protocols, and first-class delegates and user interfaces alongside contracts. The shared name reflects shared lineage, not shared design.

9 Conclusion

The architecture in this paper rests on a small number of design commitments. Applications define the algebra of their state, packaged as sandboxed WebAssembly, rather than choosing from a fixed catalogue of CRDTs. Synchronization is parameterized by that same algebra through a contract-supplied summary/delta protocol, so the wire cost of reconciliation tracks application semantics rather than raw state size. The network underneath is a small-world ring whose long-range link distribution arises from ordinary request traffic and whose forwarding decisions are informed by a per-neighbor performance model rather than topological distance alone. Private state and sensitive operations live in delegates: actor-style agents that hold secrets behind a platform-enforced trust boundary, leaving public state to contracts.

This set of commitments targets a class of decentralized applications that has been awkward to build on either content-addressed-immutable or globally-linearizable substrates: group chat, collaborative documents, social feeds, reputation systems. We have been explicit about which mechanisms are deployed, which remain experimental, and which are open problems. The next iteration of this work should address those open problems, and the asymptotic scaling claims in particular, with measurements rather than design.

References

- Deepthi Devaki Akkoorath, Alejandro Z. Tomsic, Manuel Bravo, Zhongmiao Li, Tyler Crain, Annette Bieniusa, Nuno Preguiça, and Marc Shapiro. Cure: Strong semantics meets high availability and low latency. In *Proceedings of the 36th IEEE International Conference on Distributed Computing Systems (ICDCS)*, pages 405–414, 2016.
- Paulo Sérgio Almeida, Ali Shoker, and Carlos Baquero. Delta state replicated data types. *Journal of Parallel and Distributed Computing*, 111:162–173, 2018.
- Richard E. Barlow, D. J. Bartholomew, J. M. Bremner, and H. D. Brunk. *Statistical Inference Under Order Restrictions: The Theory and Application of Isotonic Regression*. Wiley, 1972.
- Juan Benet. IPFS — content addressed, versioned, P2P file system, 2014. arXiv:1407.3561.
- Christopher Berner. redb: An embedded key-value database in pure Rust, 2026. <https://github.com/cberner/redb>, accessed 2026-05-24.
- Bytecode Alliance. Wasmtime: A standalone runtime for WebAssembly, 2026. <https://wasmtime.dev/>, accessed 2026-05-24.
- Ian Clarke, Oskar Sandberg, Brandon Wiley, and Theodore W. Hong. Freenet: A distributed anonymous information storage and retrieval system. In *Designing Privacy Enhancing Technologies: International Workshop on Design Issues in Anonymity and Unobservability*, pages 46–66. Springer, 2001.

- Jan de Leeuw, Kurt Hornik, and Patrick Mair. Isotone optimization in R: Pool-adjacent-violators algorithm (PAVA) and active set methods. *Journal of Statistical Software*, 32(5):1–24, 2009.
- Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Voshall, and Werner Vogels. Dynamo: Amazon’s highly available key-value store. In *Proceedings of the 21st ACM SIGOPS Symposium on Operating Systems Principles (SOSP)*, pages 205–220, 2007.
- Bryan Ford, Pyda Srisuresh, and Dan Kegel. Peer-to-peer communication across network address translators. In *USENIX Annual Technical Conference*, pages 179–192, 2005.
- Jon Kleinberg. The small-world phenomenon: An algorithmic perspective. In *Proceedings of the Thirty-Second Annual ACM Symposium on Theory of Computing (STOC)*, pages 163–170, 2000.
- Petar Maymounkov and David Mazières. Kademlia: A peer-to-peer information system based on the XOR metric. In *International Workshop on Peer-to-Peer Systems (IPTPS)*, pages 53–65. Springer, 2002.
- Antony Rowstron and Peter Druschel. Pastry: Scalable, decentralized object location, and routing for large-scale peer-to-peer systems. In *Middleware 2001*, pages 329–350. Springer, 2001.
- Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *Proceedings of the 13th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS)*, pages 386–400. Springer, 2011.
- Ion Stoica, Robert Morris, David Karger, M. Frans Kaashoek, and Hari Balakrishnan. Chord: A scalable peer-to-peer lookup service for internet applications. In *Proceedings of ACM SIGCOMM*, pages 149–160, 2001.
- Douglas B. Terry, Marvin M. Theimer, Karin Petersen, Alan J. Demers, Mike J. Spreitzer, and Carl H. Hauser. Managing update conflicts in Bayou, a weakly connected replicated storage system. In *Proceedings of the Fifteenth ACM Symposium on Operating Systems Principles (SOSP)*, pages 172–182, 1995.
- The Freenet Project. Ghost Keys: Anonymous, verifiable identities on Freenet, 2026a. <https://freenet.org/ghostkey/>, accessed 2026-05-24.
- The Freenet Project. River: A decentralized group chat on Freenet, 2026b. <https://github.com/freenet/river>, accessed 2026-05-24.
- The Matrix.org Foundation. The Matrix specification, 2026. <https://spec.matrix.org/>, accessed 2026-05-24.
- Tokio Project. Turmoil: A framework for testing distributed systems, 2026. <https://docs.rs/turmoil>, accessed 2026-05-24.
- Andrew Tridgell. *Efficient Algorithms for Sorting and Synchronization*. PhD thesis, The Australian National University, 1999.
- Gavin Wood. Ethereum: A secure decentralised generalised transaction ledger, 2014. Ethereum Project Yellow Paper.